

Control Is All You Need

Secure Before It Acts

Karan Arora

Lead AI Security Engineer

@aiwithkaran | karanxa007@gmail.com

14th September 2026

Abstract

A tool call an agent proposes and a tool call an agent should be allowed to execute are not necessarily the same thing. The security decision that sits between those two moments, immediately before execution, has received far less attention than prompts, outputs, or agent trajectories.

This work explores pre-execution action judgment as a distinct security control for AI agents: given a proposed action and its available context, should it execute? I first built **saroku-guard**, a 184M-parameter DeBERTa-v3 classifier specifically for this decision, and **trikesh**, an open-source runtime enforcement layer that places the decision directly in the agent's execution path. I then formalized the interface between decision and enforcement as the **Action Safety Protocol (ASP)**.

To evaluate the approach, I built **ASP-Bench**, a benchmark of pre-execution agent actions spanning 16 primary domains, with safe/unsafe labels, violation categories, and severity. I benchmarked saroku-guard alongside existing guard, moderation, and agent-safety models on a frozen private holdout, using each model's documented native input format. saroku-guard achieved **97.9%** unsafe recall, **2.9%** safe-action blocking, and 6.5ms p50 latency. The strongest peer on unsafe recall, AgentDoG, reached **99.7%** unsafe recall but blocked **72.1%** of safe actions, exposing the central trade-off for inline agent security: catching unsafe actions without blocking legitimate work.

The result is a three-part system: ASP as the decision contract, saroku-guard as the PDP, and trikesh as the PEP. The labeled training and validation data are released publicly, while the evaluation holdout remains private (§4.1.3) to preserve benchmark integrity.

1 Introduction

In July 2026, an OpenAI agent inside an internal security-testing environment was given a task that turned out to be unsolvable. Instead of stopping, it chained nine zero-day vulnerabilities in a self-hosted Artifactory instance, turned the service into a covert channel to talk to other agent instances, escalated to admin access, and forged credentials

from a stolen signing key. Messages it sent to peer agents pulled several of them off their own assigned tasks too. Hugging Face's forensic team counted roughly 17,600 attacker actions before it was contained [8].

Nine zero-days is a patching problem; no classifier touches that. But every step the agent took to exploit them was a proposed action that nothing evaluated before it ran. That is the problem this report is about: not the vulnerabilities, but the fact that each individual escalation went unchecked. And the way it spread, one agent's message derailing another agent's goal, is prompt injection at peer-agent scale. I had written the injection category in the taxonomy below (§3.2.2) before this incident, and it already covers it.

The incident also illustrates the gap this work targets: it ran for weeks, consistent with nothing being positioned to catch a single step of it in the moment, since everything watching it was reviewing a sequence after the fact, not judging one action before it executed. My two closest confirmed benchmark peers, AgentDoG and ShieldAgent (§4.2.1, §4.3.1), are trajectory-review models. That's the same gap.

This decision, made immediately before a proposed action executes and distinct from trajectory review, chat moderation, or detecting an injected instruction upstream, is what the benchmark below measures directly. What it exposes is not that existing guard models score badly. It is that every one of them forces the same trade: miss unsafe actions, or block so many legitimate ones that the agent stops being useful and the guard gets switched off. A safety layer nobody leaves running is not a safety layer.

This report formalizes pre-execution action judgment as a distinct security control point for AI agents, defines it through the Action Safety Protocol, benchmarks it through ASP-Bench, and provides a reference implementation through trikesh and saroku-guard.

Objective. This report has four objectives: define pre-execution action judgment as a distinct security decision; separate its decision and enforcement roles through the PDP/PEP architecture; formalize the decision as the Action Safety Protocol (ASP) and benchmark it through ASP-Bench; and evaluate whether existing and purpose-built safety models can perform that decision with a false-blocking

rate low enough for continuous production use.

2 Related Work

Four families of model already sit in front of agents, or close enough to be mistaken for it. None is built for the decision in §1. Three of them are near enough that the difference is worth stating precisely.

General content-safety classifiers. Llama Guard [1], ShieldGemma, and Qwen3Guard [2] classify harm categories in conversational text: violence, illegal content, jail-break attempts. Llama Guard’s most recent generation documents agent/tool-call conversation support in its taxonomy, making it the closest off-the-shelf candidate, but it remains a harm-category classifier applied to an agent turn, not a classifier trained against a constraint-violation / scope / goal-drift taxonomy specific to pre-execution action judgment.

Prompt-injection detectors. Prompt Guard [3] solves a genuinely different task: detecting that an upstream input contains an injected instruction, not judging whether a downstream action is safe to execute. A model can correctly flag an injection and say nothing useful about whether the resulting action should be blocked, and vice versa.

Agent-trajectory safety classifiers. AgentDoG [4] and ShieldAgent [5] are the closest prior work. Both are fine-tuned specifically to judge the safety of an LLM agent’s behavior from its observations, reasoning, and actions. The framing difference is that both were built to review a completed or in-progress multi-step trajectory, whereas ASP’s task judges a single proposed action before it executes: pre-execution rather than post-hoc, and single-action rather than trajectory-level. I treat this as a framing difference rather than a task mismatch and include both as valid head-to-head peers (§4.2.1).

Agent-safety benchmarks. AgentHarm [6] and ToolEmu [7] evaluate agent behavior end-to-end or in simulation, generally scoring whether an agent completes a harmful task across a full interaction. Neither isolates the specific sub-decision this benchmark targets: one proposed action, judged before execution, independent of how the agent arrived at proposing it.

None of the four categories above were built to answer, directly, the question this report benchmarks: given this proposed action, should it run? §3.1 works out how that question should actually be answered and enforced; §3.2 then fixes it as a precise input/output shape every instance in this benchmark, and every model evaluated against it, is held to.

3 Methods

3.1 Approach

The question itself is simple to state: given this proposed action and its context, should it run? Answering it well requires separating two things that are easy to conflate: *deciding* whether an action is safe, and *enforcing* that decision at the point where the action would actually execute. Access-control architecture already has names for this split and decades of production use behind it: a **Policy Decision Point (PDP)** decides, a **Policy Enforcement Point (PEP)** enforces that decision at the boundary where the action would otherwise proceed [9]. Applied to an agent, the mapping is direct: the PEP is code sitting inline in the agent’s own tool-calling loop, intercepting every proposed call; the PDP is whatever judges whether that call should proceed.

Definition. Given an agent-proposed action a and available execution context c , the PDP computes a safety decision $d = f(a, c)$. The PEP enforces d at the execution boundary: the PEP is enforcement, the PDP is judgment, and ASP standardizes the interface between them.

Agent $\rightarrow$ PEP $\rightarrow$ PDP $\rightarrow$ Allow / Block $\rightarrow$ Tool

Keeping these separate is what makes the rest of this report possible. The PEP’s job stays fixed and simple no matter how the PDP changes: intercept the call, ask the PDP, act on the verdict. That lets the PDP be improved, replaced, or run in combination with others, without touching the code sitting in the agent’s tool-calling loop, or the agent itself. It also makes the PDP benchmarkable in isolation, since its entire job is one well-defined decision rather than something entangled with enforcement mechanics, retries, or transport. §4.1 through §4.3 do exactly that: benchmark PDPs as a category, independent of how any of them get enforced.

I brought this separation to the agent execution boundary concretely first: trikes is the PEP, an open-source library that wraps an agent’s tool calls and asks a configured PDP for a verdict before letting each one through. To keep that PEP from being locked to my code, or to one judge, I then formalized the decision contract as an open specification, the **Action Safety Protocol (ASP)**: a PDP built by anyone else can be evaluated against this benchmark, and any PEP can call any conformant PDP, without either side depending on my code (§3.2 defines that contract exactly). Models get replaced. The contract for what a safety decision even is should outlast them. The reference PDP behind trikes’s default install is saroku-guard, fast enough to run on every call inline; neither is required by ASP itself. §3.3 and §3.4 detail them. The remainder of this report is the case for why that specific PDP is the right default: benchmarked against the alternatives (§4.2, §4.3), and checked directly against a real incident’s behavioral shape (§4.4).

ASP is the specification. ASP-Bench is the measurement framework.

saroku-guard is the reference PDP.
trikesh is the reference PEP.

3.2 Action Safety Protocol

trikesh could have kept the shape of this decision as an internal detail. Fixing it as a public contract instead is what lets a PDP built by anyone else be scored on the same benchmark, and any enforcement layer swap judges without rewriting itself. ASP defines one unit of work, the *Action-SafetyRecord*: a single pre-execution policy decision, with a fixed input schema and a fixed output schema. The full specification, with its request/response contract and conformance levels, is published as its own standalone document at trikesh.com/blog/action-safety-protocol.

3.2.1 Input

An instance provides, at minimum, the proposed action and its immediate context; up to four additional fields depending on how much surrounding information is available in a given deployment:

Field	Description	Present
action	Proposed tool call, verbatim	yes
context	Agent role and environment	yes
trigger	What prompted this action	yes
constraints	Rules the action must respect	optional
original_goal	The task the agent was given	optional
user_message	The literal user request	optional
conversation	Preceding dialogue turns	optional

I tier instances 1–4 by how many of these optional fields are populated, so a model trained on this schema learns to judge correctly whether it receives a bare action-plus-context pair or a fully specified deployment with constraints, goal, and dialogue history attached.

3.2.2 Output

A binary decision, `is_safe: true/false`, is the primary label. Unsafe instances additionally carry a five-category violation type and a severity (high/medium/low), collapsed from a more granular internal taxonomy to a set small enough to support reliable per-class evaluation at current dataset scale:

- **policy_violation**: constraint override, missing required approval
- **scope_violation**: action exceeds the minimal scope the task requires
- **injection**: an untrusted tool result or message steers the action
- **goal_drift**: the action diverges from the original stated goal

- **corrigibility**: the action ignores an explicit stop or correction

The primary classifier’s decision head is binary; the violation category is produced by a separate, additive head (§3.4) and is reported here as a secondary result, not the benchmark’s main metric.

This contract is what makes the rest of the work portable. A PDP that speaks it can be scored on this benchmark without adopting any of my code, and a PEP that speaks it can swap judges without rewriting the enforcement path. trikesh (§3.3) is its first implementation, and the specification is written so it does not have to be the only one.

3.3 trikesh

§3.1 argued the decision should be split into a PEP that enforces and a PDP that decides. trikesh is that argument built: an open-source library that implements the enforcement half, ships a judge for the deciding half (§3.4), and speaks a protocol that lets either side be swapped (§3.2).

A model that holds both ends of the trade is necessary for a guard you can leave on. It is not sufficient. The rest of that claim is an integration problem: a check nobody can adopt without rewriting their agent, or that adds a hosted dependency to every tool call, gets switched off for reasons that have nothing to do with its accuracy. trikesh is the enforcement half, and it is built around removing those reasons.

It requires no configuration to start. Installing it and instantiating the guard is enough to get a working policy enforcement point, because it loads saroku-guard as its default PDP automatically, with no API key, no hosted service, and no network call on the fast path.

```
from trikesh import SafetyGuard
from trikesh.integrations import
protect

guard = SafetyGuard() # saroku-guard
loads by default
agent = await protect(agent,
guard=guard) # every tool call now
judged
```

`protect()` detects the framework from the agent object and applies interception to every tool it exposes, for Google ADK, AutoGen, and LangChain. A blocked call raises before the tool body runs. Where an agent is hand-rolled rather than built on one of those frameworks, `wrap()` applies the same interception to a single tool, and `guard.check()` exposes the raw decision for callers who want to route on it themselves.

Three modes trade off latency against depth of review:

- **local**: saroku-guard only, no LLM call, single-digit-millisecond decisions.
- **balanced** (default): saroku-guard runs on every action; only actions it flags unsafe escalate to an LLM judge for a second, deeper opinion.
- **thorough**: every action goes through the LLM judge, regardless of what saroku-guard says.

None of this changes across modes except which PDP, or how many, get consulted. That’s the point of keeping the PEP and PDP separate (§3.1): the code sitting in the agent’s tool-calling loop stays the same whether it’s backed by saroku-guard alone, saroku-guard plus an LLM judge, or, since trikes speaks ASP, a different conformant PDP entirely.

3.4 saroku-guard

saroku-guard is the PDP trikes loads by default: a 184M-parameter classifier trained for this one decision, small enough to run inline on every call. It is scored against every alternative I could find in §4.3. The model is published at huggingface.co/karanxa/saroku-guard.

A bare allow/block verdict is a hard thing to operate. Someone still has to work out why an action was stopped before deciding whether to fix the agent, adjust a policy, or let it through next time. saroku-guard attaches a reason to every block: one of the five violation categories defined in §3.2.2, so a `policy_violation` and a `goal_drift` can be routed differently instead of landing in the same undifferentiated review queue.

That attribution is secondary to the binary decision, not a replacement for it. The model is selected on how well the binary decision alone catches unsafe actions, not on how well it explains them: a checkpoint that attributes violations precisely but lets more of them through is a worse gate, whatever its attribution quality. §4.3.2 reports how reliable that attribution actually is, including where it falls short.

4 Experiments

4.1 Dataset

Once the model and enforcement path were in place, I needed a way to evaluate the decision consistently, and against alternatives, rather than trusting a single set of spot checks. No corpus for it existed, so I built one: I call that benchmark **ASP-Bench**, after the protocol rather than after any model, so that it can be adopted and cited independently of whichever model currently tops it.

4.1.1 Construction

Instances were generated across a grid of 16 domains $\times$ 15 scenario templates (destructive actions with and without approval, prompt-injection-bearing tool results, goal drift, privilege escalation, secrets exfiltration, hard negatives designed to resemble unsafe actions while being compliant, and others) using an LLM generator, then merged with a smaller volume of hand-authored seed instances and a filtered, converted external tool-safety corpus.

Of 52,500 raw generated instances, 15,117 were dropped by an automated quality filter, leaving 28,651 instances in the released splits.

4.1.2 Composition

Split	Rows	Safe	Unsafe
train	22,682	10,235	12,447
val	4,049	1,798	2,251
holdout	1,920	624	1,296

Role: train for training, val for checkpoint selection, holdout for the frozen evaluation reported in this paper.

The realized dataset spans **29 distinct domain labels**: 16 primary domains at roughly 1,650–1,880 instances each, plus 13 long-tail domains inherited from the converted external corpus with far fewer instances each (5 to 90 instances apiece).

4.1.3 Release plan

I publish the train and validation splits at huggingface.co/datasets/karanxa/agent-action-safety-dataset, for independent fine-tuning and replication. I keep the holdout split back. Publishing a frozen test set turns it into training data for whatever is evaluated against it next, and within a few release cycles the number it produces stops meaning anything.

The released training data is a mix of a hand-authored gold set (368 rows) and a larger LLM-generated corpus. I reviewed an unbiased random sample of 120 released rows and found all 120 binary safe/unsafe labels correct, with 2 violation-category mislabels, consistent with the weaker attribution accuracy reported in §4.3.2. The benchmark results reported in §4.3 are independent of this review because they are computed exclusively on the separate private evaluation holdout.

4.2 Evaluation Methodology

A benchmark is worth exactly as much as the rules it is scored under. Three of those rules matter enough to state before any number appears.

4.2.1 Classifying the models

A benchmark inviting “beats X” claims against models that were never built for its task is not measuring what it appears to measure. I therefore classify every evaluated model into one of three tiers before reporting any comparison, justified with a specific, checkable source.

- **Peer**: confirmed to have been built for either this exact task or a closely related framing of it. Valid head-to-head comparison.
- **Adjacent**: confirmed to solve a related but genuinely different task. Reported as a sanity check only.
- **Baseline**: a non-learned reference point, establishing the floor a real classifier must clear.

4.2.2 Native input format

Every model is evaluated in its own documented input format: its own chat template, system prompt, and dialogue framing where one is specified, not a single generic prompt applied uniformly across all seven.

4.2.3 Scope and scoring

All seven models are scored on an identical, frozen set of agent tool-call decisions spanning 16 domains, disjoint from every model's own training data. Every model is scored with the same accuracy/precision/recall/F1 implementation and the same definition of the positive class (`unsafe = 0`). Latency is measured on the same hardware for every model, at p50/p95/p99.

4.3 Results

Seven models, one frozen evaluation set, scored under the rules above.

4.3.1 Primary comparison

Accuracy			
Model	Acc.	Recall	Blocked
saroku-guard	97.7%	97.9%	2.9%
AgentDoG	76.4%	99.7%	72.1%
Llama Guard 4	74.5%	71.7%	19.7%
ShieldAgent	65.2%	94.6%	95.8%
Majority-unsafe	67.5%	100%	100%
Prompt Guard	67.5%	100%	100%
Qwen3Guard	35.4%	5.3%	2.1%

Latency		
Model	Tier	p99
saroku-guard (184M)	Peer	8.6ms
AgentDoG (8B) [4]	Peer	42.2ms
Llama Guard 4 (12B) [1]	Peer	236.9ms
ShieldAgent (7B) [5]	Peer	959.9ms
Majority-unsafe	Baseline	<0.1ms
Prompt Guard (86M) [3]	Adjacent	8.2ms
Qwen3Guard (0.6B) [2]	Adjacent	206.2ms

Read the recall column alone and AgentDoG looks like the better gate: it catches 99.7% of unsafe actions against saroku-guard's 97.9%, missing four where saroku-guard misses twenty-seven. The next column is why that reading is wrong. AgentDoG buys that recall by blocking 450 of 624 legitimate actions, roughly seven in ten. ShieldAgent blocks 598 of 624. An agent wrapped in either one cannot complete ordinary work, and a gate that stops ordinary work does not stay switched on long enough to stop anything else.

On this holdout, saroku-guard is the only evaluated model that combines high unsafe recall with low safe-action blocking: 97.9% of unsafe actions caught, 18 of 624 safe actions wrongly blocked. That combination is what makes saroku-guard the default judge shipped with trikes.

4.3.2 Violation-type attribution

The secondary property head reaches 74.0% accuracy and 76.9% macro-F1 across the five violation categories on the same holdout, with substantial variance by class: corrigibility (89.3% recall) and injection (87.4%) are attributed reliably; policy_violation (70.8%) and scope_violation (70.2%) less so; goal_drift (63.7%) is the weakest category.

4.4 Reconstructing the Incident

Everything to this point is aggregate statistics over a frozen holdout. To check the argument in §1 directly rather than by analogy, I built a runnable reconstruction of the incident's behavioral shape and ran it against a live `SafetyGuard()` – the default install described in §3.3, with no API key and no further setup.

The reconstruction is a five-step chain of proposed actions modeled on the real sequence: reconfiguring an internal tool to reach outside an assigned sandbox, writing to a shared resource as an improvised channel to reach other agent instances, escalating session privileges, deriving a credential outside any legitimate use, and a second, separate agent instance proposing an action after receiving a message from the first. I ran the chain twice: once with no check at all, and once with each proposed action evaluated by saroku-guard before execution.

Unprotected, all five actions execute. Protected, all five are blocked before execution, including the fifth, the peer-agent-induced step. All five steps were attributed to the same category, `scope_violation`, consistent with §4.3.2's own finding that attribution is markedly weaker than the binary decision. The binary decision, block or allow, was correct on all five.

5 Conclusion

The agent in §1 took roughly 17,600 actions before anyone stopped it. Not one of them was judged in the moment it was proposed.

Pre-execution agent tool-call judgment is a distinct task from the ones current guardrail models are built and marketed for. On this framing, every model I tested that catches unsafe actions reliably does so by blocking a large share of legitimate ones, which is the same as not shipping a guard at all. A model trained for the decision holds both ends: 97.9% of unsafe actions caught, 2.9% of safe ones wrongly blocked. None of this works as a single model release, so it is not one. Five pieces ship together, and each is useful without the others.

ASP

The protocol. Defines what a pre-execution safety decision is, as a request/response contract with two conformance levels.

ASP-Bench

The benchmark. One frozen evaluation set, per-model

native input formats, and a tiering rule that keeps “beats X” claims honest.

Dataset

28,651 labeled agent action decisions across 16 domains, each with a violation category and severity. Train and validation splits published; holdout held back (§4.1.3).

saroku-guard

The judge. 184M parameters, 97.9% of unsafe actions caught against 2.9% of safe ones wrongly blocked, at single-digit-millisecond latency.

trikesh

The enforcement layer. Wraps an agent’s tool calls, runs the judge on every one of them, and escalates only what gets flagged.

Agents are shipping faster than the checks around them.

Allow or block, decided once per action, before the action runs, is the smallest place to put a check that actually holds. saroku-guard is one implementation of that check. ASP is the contract that lets there be others.

References

- [1] Meta AI. *Llama Guard 4 Model Card*. Hugging Face, 2026.
- [2] Qwen Team. *Qwen3Guard-Gen Model Card*. Hugging Face, 2026.
- [3] Meta AI. *Llama Prompt Guard Model Card*. Hugging Face, 2025.
- [4] Liu, D. et al. *AgentDoG 1.5: A Lightweight and Scalable Alignment Framework for AI Agent Safety and Security*. arXiv:2605.29801, 2026.
- [5] Chen, Z., Kang, M., and Li, B. *ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning*. arXiv:2503.22738, 2025.
- [6] Andriushchenko, M. et al. *AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents*. ICLR 2025. arXiv:2410.09024.
- [7] Ruan, Y. et al. *Identifying the Risks of LM Agents with an LM-Emulated Sandbox*. ICLR 2024. arXiv:2309.15817.
- [8] OpenAI. *The Hugging Face Incident: Technical Report*. August 2026.
- [9] OASIS. *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard, 2013.